



Universidad
Nacional de
General
Sarmiento

Instituto de Industria
Licenciatura en Sistemas

TRABAJO FINAL

Sistemas Operativos y Redes 2

Primer Semestre 2026

"Simulación y Análisis Comparativo de Redes P2P Estructuradas (Kademlia/DHT) y No Estructuradas (Flooding Gnutella)"

Alumno/a	Lautaro Manuel Avalos
Legajo	42831243
Docente	Benjamin Chuquimango
Fecha de entrega	24/06/2026

Modalidad Individual — Obligatorio

Buenos Aires, Argentina — 2026

Índice de contenidos.....	2
1. Resumen ejecutivo.....	4
2. Planteamiento del problema.....	5
2.1 El problema de localización en redes P2P a escala.....	5
2.2 Relevancia académica y motivación del trabajo.....	5
3. Pregunta de investigación y objetivos.....	6
3.1 Pregunta de investigación.....	6
3.2 Objetivo general.....	6
3.3 Objetivos específicos.....	6
4. Marco teórico.....	7
4.1 Arquitecturas de redes P2P.....	7
4.2 Kademlia y la métrica XOR.....	7
4.3 Fórmulas de distribución P2P.....	8
4.4 Robustez bajo churn.....	8
4.6 Estado del arte: Kademlia en sistemas P2P contemporáneos.....	9
5. Metodología.....	9
5.1 Enfoque metodológico.....	9
5.2 Herramientas y justificación.....	10
5.3 Parámetros experimentales.....	11
6. Implementación y desarrollo.....	12
6.1 Diseño de la solución.....	12
Arquitectura del simulador.....	12
Decisiones de diseño principales.....	13
6.2 Implementación por componente.....	14
Módulo node.py.....	14
Módulo network.py.....	14
Módulo flooding.py.....	14
Módulo kademlia.py.....	15
Inicialización de k-buckets:.....	15
Bootstrap del nodo:.....	15
Búsqueda iterativa:.....	15
Módulo simulation.py.....	16
CLI de orquestación:.....	16
Módulo metrics.py.....	17
6.3 Análisis según modelo OSI.....	18
Capa 7 — Aplicación.....	18
Capa 4 — Transporte.....	19
Capa 3 — Red.....	19
Capas 1–2 — Enlace y Física.....	20
6.4 Pruebas y verificación.....	20
7. Resultados y análisis.....	22
7.1 Flooding: verificación de $O(N)$	22

Datos experimentales.....	22
Validación $O(N)$ mediante regresión log-log.....	22
Varianza y comportamiento bimodal.....	23
Independencia de la latencia respecto de N	23
7.2 Kademia: verificación de $O(\log N)$	24
Datos experimentales.....	24
Validación $O(\log N)$ mediante regresión lineal.....	24
Baja tasa de éxito en redes pequeñas.....	25
Factor de diferencia entre arquitecturas.....	25
7.3 Validación contra fórmulas $Tcs/Tp2p$	26
Parámetros del cálculo.....	26
Resultados.....	26
Correspondencia de clases de complejidad.....	27
Tabla comparativa flood vs Kademia vs overhead total.....	28
7.4 Análisis de churn y robustez.....	28
Tasa de churn umbral.....	29
El hallazgo inesperado: degradación comparable en ambas arquitecturas.....	29
7.5 Comparación con los objetivos.....	30
7.6 Limitaciones identificadas.....	31
7.7 Propuestas de mejora y trabajo futuro.....	31
8. Conclusiones y trabajo futuro.....	32
8.1 Respuesta a la pregunta de investigación.....	32
8.2 La analogía $Tcs/Tp2p$	32
8.3 El hallazgo de churn.....	32
8.4 Contribución del trabajo.....	32
9. Referencias bibliográficas.....	34
10. Anexos.....	35
Anexo A — Estructura del repositorio y código fuente.....	35
Anexo B — Instrucciones de ejecución.....	35
Instalación:.....	35
Ejecución de simulaciones:.....	35
Análisis y gráficos:.....	36
Tests unitarios:.....	36
Anexo C — Análisis de sensibilidad de parámetros.....	36
Sensibilidad K en flooding.....	36
Saturación del espacio de IDs con $B = 8$	37
Distribución de carga por nodo.....	38

1. Resumen ejecutivo

Las redes peer-to-peer (P2P) constituyen la infraestructura de sistemas críticos como BitTorrent, IPFS y Ethereum. El problema central de cualquier red P2P es la localización de recursos: ¿cuántos mensajes necesita intercambiar la red para encontrar un archivo específico, y cómo escala ese número con el tamaño de la red?

Este trabajo implementa y compara en Python dos arquitecturas P2P: flooding no estructurado (tipo Gnutella) y Kademlia DHT estructurado, para $N \in \{10, 50, 100, 500, 1.000, 5.000, 15.000\}$ nodos y 100 repeticiones por configuración. Se midieron mensajes por búsqueda, saltos hasta el recurso y tasa de éxito bajo churn (5%, 10%, 20%), y los resultados se validaron contra las fórmulas teóricas de tiempo de distribución T_{cs} y T_{p2p} estudiadas en la Unidad 9.

Los resultados principales confirman las predicciones teóricas con ajuste $R^2 > 0,99$ en ambos casos: flooding exhibe crecimiento lineal (pendiente log-log = $1,019 \approx 1$) con constante empírica $0,78 \times N$, mientras que Kademlia sigue $\text{hops} = 0,473 \times \log_2(N) - 0,636$. A $N = 15.000$ nodos la diferencia es de un factor $\times 2.247$ en favor de Kademlia. La comparación con las fórmulas T_{cs}/T_{p2p} revela que flooding y T_{cs} pertenecen a la misma clase de complejidad $O(N)$ —aunque por razones distintas—, y que Kademlia y T_{p2p} son ambos sub-lineales, estableciendo una analogía formal entre las dos dimensiones del problema (búsqueda y transferencia). Bajo churn, la degradación en ambas arquitecturas está dominada por pérdida de contenido —no por fallos de ruteo—, consecuencia documentada del modelo sin replicación.

Palabras clave: *redes P2P, Kademlia, DHT, flooding, Gnutella, distancia XOR, churn, simulación discreta, $O(\log N)$, T_{cs}/T_{p2p}*

2. Planteamiento del problema

2.1 El problema de localización en redes P2P a escala

Las redes peer-to-peer permiten distribuir recursos (archivos, datos, servicios) entre millones de participantes sin servidor central. Sin embargo, la ausencia de un directorio global plantea una pregunta de diseño fundamental: ¿cómo encuentra un nodo el recurso que necesita cuando ningún participante tiene una vista completa de la red?

La solución más directa, el flooding, hace que cada nodo pregunte a todos sus vecinos, quienes a su vez preguntan a los suyos. Este mecanismo garantiza encontrar cualquier recurso existente, pero genera un tráfico proporcional al número de nodos de la red. Con $N = 1.000$ nodos, una sola búsqueda genera ~ 900 mensajes; con $N = 1.000.000$, genera ~ 780.000 mensajes. A escala de BitTorrent —que en su pico de actividad superó los 150 millones de usuarios activos (Cohen, 2008)— el flooding hubiera generado del orden de 10^8 mensajes por búsqueda, haciendo inviable cualquier actividad en la red.

Este fue exactamente el colapso que experimentó Gnutella en 2001, red que utilizaba flooding puro: el tráfico de consultas llegó a representar el 50% del tráfico total de la red, degradando el servicio a medida que crecía la base de usuarios (Ripeanu et al., 2002). El problema no era técnico sino estructural: la complejidad de búsqueda del flooding es $O(N)$, y el crecimiento de la red lo hacía inevitable.

Kademlia (Maymounkov & Mazières, 2002) resolvió este problema con una idea elegante: organizar los nodos en un espacio de identificadores de B bits y usar la distancia XOR para asignar responsabilidades. Una búsqueda converge en $O(\log N)$ saltos porque cada salto reduce la distancia al objetivo a la mitad. A $N = 1.000$ nodos, Kademlia necesita ≈ 4 mensajes donde el flooding necesita 900: un factor $\times 236$.

La adopción de Kademlia fue inmediata y permanente. BitTorrent incorporó Mainline DHT (basado en Kademlia) en la versión 4.2.0 (2005). IPFS, el sistema de archivos distribuido que aspira a reemplazar al HTTP, usa una variante de Kademlia como capa de descubrimiento (Trautwein et al., 2022). Ethereum, la plataforma de contratos inteligentes, usa una DHT derivada de Kademlia para el descubrimiento de nodos en su red DevP2P.

2.2 Relevancia académica y motivación del trabajo

Las fórmulas T_{cs} y T_{p2p} estudiadas en Unidad 9 modelan el tiempo de distribución de archivos en redes cliente-servidor y P2P, respectivamente. Estas fórmulas son análisis teóricos sobre papel. Una pregunta natural es: ¿qué tan bien predicen el comportamiento de una simulación real? ¿Bajo qué condiciones sus supuestos simplificadores —red estática, uploads homogéneos, latencia cero— producen predicciones precisas, y cuándo divergen?

Este trabajo convierte esos cálculos teóricos en una simulación que permite responder esas preguntas con evidencia empírica. La contribución no es solo confirmar la teoría,

sino identificar con precisión las condiciones donde los supuestos simplificadores fallan y cuantificar la magnitud de esa divergencia.

3. Pregunta de investigación y objetivos

3.1 Pregunta de investigación

¿En qué medida los mensajes por búsqueda de una simulación de flooding en Python ($N = 10$ a 15.000 , $K = 10$) se aproximan a la predicción $O(N)$ de la teoría, y cómo se compara con los saltos $O(\log N)$ de Kademlia ($B = 16$ bits) bajo tasas de churn de 5%, 10% y 20%, identificando las condiciones donde los supuestos simplificadores del modelo teórico producen divergencias medibles?

3.2 Objetivo general

Validar experimentalmente las predicciones teóricas de complejidad $O(N)$ para flooding y $O(\log N)$ para Kademlia mediante simulación en Python, comparar ambas arquitecturas en eficiencia de búsqueda y robustez bajo churn, y relacionar los resultados con las fórmulas de distribución $Tcs/Tp2p$ estudiadas en la Unidad 9 del programa.

3.3 Objetivos específicos

1. Implementar y validar una simulación de red P2P no estructurada (flooding tipo Gnutella) con $N \in \{10, 50, 100, 500, 1.000, 5.000, 15.000\}$ nodos y $K = 10$ vecinos, verificando que los mensajes por búsqueda crecen como $O(N)$ con pendiente ≈ 1 en escala log-log.
2. Implementar y validar una simulación de Kademlia DHT con $B = 16$ bits, k -buckets de tamaño $k = 3$ y búsqueda iterativa, verificando que los saltos por búsqueda crecen como $O(\log N)$ con $R^2 > 0,95$.
3. Calcular los valores de Tcs y $Tp2p$ según las fórmulas del programa para los mismos valores de N y establecer la correspondencia de clases de complejidad entre la simulación (Fase 1: búsqueda) y las fórmulas (Fase 2: transferencia).
4. Simular churn en tasas de 5%, 10% y 20% para 20 rondas de 50 búsquedas cada una, medir la tasa de éxito en ambas arquitecturas, e identificar el umbral de degradación significativa ($>10\%$ de fallos).
5. Documentar y analizar las divergencias entre simulación y teoría, identificando qué supuestos simplificadores producen cada divergencia y qué extensiones del modelo serían necesarias para eliminarlas.

4. Marco teórico

4.1 Arquitecturas de redes P2P

Las redes peer-to-peer se clasifican en no estructuradas y estructuradas según el mecanismo de organización de los participantes (Kurose & Ross, 2021, cap. 2).

Redes no estructuradas. En una red no estructurada cada nodo mantiene conexiones con un conjunto arbitrario de vecinos, sin garantía de dónde está almacenado un recurso dado. La búsqueda se realiza por inundación (flooding): el nodo iniciador envía la consulta a todos sus vecinos, quienes la propagan a los suyos si no la recibieron antes. El proceso termina cuando el recurso es encontrado o cuando se agota el tiempo de vida del mensaje (TTL). Gnutella 0.4 fue la implementación de referencia de este modelo. La ventaja del flooding es su simplicidad y robustez conceptual: siempre encuentra el recurso si existe y la red es conexas. Su desventaja fundamental es que el número de mensajes por búsqueda es $O(N)$ (Ripeanu et al., 2002), lo que hace al modelo inescalable a redes grandes.

Redes estructuradas (DHT). En una red estructurada, nodos y recursos comparten un espacio de identificadores y el sistema mantiene invariantes que garantizan que cualquier recurso puede ser localizado consultando $O(\log N)$ nodos (Kurose & Ross, 2021). Las tablas de hash distribuidas (DHT) son la implementación canónica de este modelo.

4.2 Kademlia y la métrica XOR

Kademlia (Maymounkov & Mazières, 2002) es el protocolo DHT más ampliamente adoptado. Cada nodo tiene un identificador de B bits asignado aleatoriamente. La distancia entre dos identificadores A y B se define como la operación XOR bit a bit: $d(A, B) = A \oplus B$.

La métrica XOR tiene tres propiedades que la hacen adecuada para el ruteo distribuido:

- **Simetría:** $d(A, B) = d(B, A)$. Si A conoce que B está cerca, B sabe que A está cerca; las tablas de ruteo son mutuamente consistentes.
- **Estructura de árbol binario.** Dos nodos son XOR-cercanos cuando sus IDs comparten un prefijo binario largo. Esta propiedad impone una partición jerárquica del espacio de IDs que permite organizar las tablas de ruteo de forma eficiente.
- **Unidireccionalidad.** Para un nodo A y una distancia d , existe exactamente un ID a esa distancia: $A \oplus d$. Esto garantiza que el ruteo converge sin ciclos: cada salto avanza al menos 1 bit en la distancia XOR, reduciendo el espacio de candidatos a la mitad. En a lo sumo $\log_2(N)$ saltos se alcanza el nodo responsable del recurso.

Tablas de ruteo (k-buckets). Cada nodo mantiene B buckets de k entradas. El bucket i contiene peers cuyo ID difiere del propio en el i -ésimo bit más significativo: el bucket

1 cubre la mitad del espacio de IDs más lejana; el bucket B cubre el segmento más cercano. Con k entradas por bucket, un nodo conoce como máximo $k \times B$ peers distribuidos de forma estructuralmente balanceada en el espacio de IDs.

Búsqueda iterativa. Para encontrar la clave K, el nodo iniciador consulta a los α peers más cercanos a K que conoce. Cada peer responde con los k peers más cercanos que él conoce. El iniciador actualiza su shortlist y repite con los candidatos aún no contactados que sean más cercanos al objetivo que el mejor encontrado hasta ese momento. El proceso termina cuando ningún nuevo candidato mejora la distancia XOR.

4.3 Fórmulas de distribución P2P

Las fórmulas de tiempo de distribución de archivos modelan cuánto tarda una red en entregar un archivo de tamaño F a N destinatarios (Kurose & Ross, 2021):

$$T_{cs} = \max\left(\frac{NF}{U_s}, \frac{F}{d_{\min}}\right)$$

$$T_{p2p} = \max\left(\frac{F}{U_s}, \frac{F}{d_{\min}}, \frac{NF}{U_s + \sum U_i}\right)$$

donde U_s es el upload del servidor/seed, $d_{\min}$ es la menor velocidad de descarga entre los peers, y $\sum U_i$ es la suma de los uploads de todos los peers.

T_{cs} crece linealmente con N (cuando el término dominante es NF/U_s): el cuello de botella es el servidor, que debe enviar el archivo a cada destinatario secuencialmente. T_{p2p} crece sub-linealmente porque el denominador $U_s + \sum U_i$ aumenta con N: cada peer que recibe el archivo puede a su vez contribuir con su upload, y en el límite T_{p2p} converge a $F/\bar{U}_i$.

Estas fórmulas modelan la Fase 2 del proceso P2P (transferencia del archivo una vez localizado). La simulación modela la Fase 1 (localización del recurso). Ambas dimensiones son necesarias para evaluar el costo total de una operación P2P y se relacionan en la Sección 8.3.

4.4 Robustez bajo churn

Se denomina churn al proceso continuo de entrada y salida de nodos de una red P2P. El churn afecta de manera distinta a cada arquitectura. En flooding, la ausencia de tablas de ruteo hace que el mecanismo de búsqueda sea estructuralmente robusto: la red puede seguir ruteando mensajes aunque entren y salgan nodos, sin necesidad de mantenimiento activo. En Kademlia, los k-buckets quedan con entradas obsoletas cuando los nodos se van; el protocolo mitiga esto mediante republicación periódica de claves y actualizando los buckets al recibir cualquier mensaje entrante (Maymounkov & Mazières, 2002).

Sin embargo, la ventaja de Kademlia bajo churn depende de un mecanismo adicional: la replicación de claves en los $k = 20$ nodos XOR más cercanos. Si esa replicación está ausente, la disponibilidad de contenido cae al ritmo del churn con independencia del protocolo de ruteo. Las implicaciones de esta distinción para los resultados experimentales se discuten en la Sección 8.4.

4.6 Estado del arte: Kademlia en sistemas P2P contemporáneos

La adopción de Kademlia en sistemas de producción ha generado un cuerpo de investigación que evalúa sus propiedades bajo condiciones reales. Trautwein et al. (2022) midieron el comportamiento de IPFS —cuya capa de descubrimiento es una variante de Kademlia— en una red de producción con decenas de miles de nodos activos. Sus mediciones muestran tiempos de resolución de claves de 1 a 3 segundos bajo condiciones normales, un comportamiento de saltos consistente con la predicción $O(\log N)$, y una caída drástica en disponibilidad de contenido cuando el churn supera el 50% de nodos por hora sin replicación adecuada. Este trabajo empírico valida el valor $k = 20$ para los k -buckets y la replicación en los k nodos más cercanos como mecanismos eficaces para tolerar el churn a escala.

Los parámetros originales de Kademlia no son universalmente óptimos, y su ajuste es un área activa de investigación. Monteiro et al. (2022) propusieron una variante denominada "partitioned Kademlia" para IPFS en la que los identificadores de nodo se particionan para mejorar la localidad de las búsquedas, logrando mejoras de hasta un 40% en rendimiento respecto al Kademlia estándar para redes de más de 10.000 nodos. Kanemitsu & Kanai (2023) extendieron este enfoque con un método automático para derivar los parámetros óptimos de lookup en KadRTT —variante de Kademlia utilizada en libp2p/IPFS—, ajustando dinámicamente la concurrencia, el número de saltos y el tamaño de bucket según las condiciones de red.

El impacto del churn sobre la disponibilidad de datos distribuidos se documenta también fuera del ámbito estrictamente DHT. Imtiaz et al. (2021) analizaron el churn en la red Bitcoin —cuya capa de propagación comparte características con las redes P2P— y encontraron que el 97% de los nodos exhibe conectividad intermitente, generando un número significativamente mayor de fallos en la propagación de bloques respecto a nodos estables. Este resultado ilustra el impacto concreto del churn sobre la consistencia de datos distribuidos e informa el diseño de los experimentos de la Sección 8.4.

Desde una perspectiva comparativa más amplia, Naik & Keshavamurthy (2020) relevaron protocolos P2P de nueva generación bajo condiciones de alto churn y concluyeron que los mecanismos de mantenimiento de tablas de ruteo basados en DHT estructuradas son significativamente más robustos que los esquemas no estructurados, siempre que la replicación esté correctamente dimensionada.

5. Metodología

5.1 Enfoque metodológico

El trabajo adopta un enfoque de simulación discreta por eventos. Se justifica frente a las alternativas de la siguiente manera:

- Simulación vs. implementación real: una implementación real de Kademlia requiere red física, sockets UDP, gestión de latencias variables y descubrimiento real de nodos. El objetivo del trabajo es validar las predicciones teóricas de complejidad, no implementar un sistema de producción. La simulación permite controlar exactamente los parámetros (N , K , B , distribución de recursos) y repetir el experimento 100 veces bajo condiciones idénticas, lo que sería imposible en una red real.
- Simulación discreta vs. continua: el modelo de red es de eventos discretos: cada búsqueda es una unidad atómica de trabajo, y el churn se aplica entre rondas. Este

nivel de abstracción es adecuado para medir overhead de mensajes y tasa de éxito sin modelar latencias de red (que no afectan el conteo de mensajes).

- Python vs. lenguajes compilados: la velocidad de simulación en Python fue medida empíricamente: $N = 15.000$ con 100 repeticiones completa en ≈ 19 segundos. Para el rango de N evaluado, Python es suficiente. Un lenguaje compilado permitiría extender a $N = 50.000$ ($\approx 2,4$ minutos en Python), pero no cambiaría las conclusiones sobre las clases de complejidad.

El experimento se diseñó de forma reproducible: todos los parámetros están en `configs/simulation_config.yaml`, el código fuente está disponible en el repositorio público, y los datos de simulación están versionados en CSV.

5.2 Herramientas y justificación

Herramienta / Tecnología	Propósito en el trabajo	Justificación de la elección
Python 3.10+	Lógica de simulación de flooding y Kademia	Disponibilidad, rapidez de prototipado, librerías de análisis integradas. Simpy (eventos discretos) no fue necesario dado el modelo basado en rondas
NetworkX 3.0+	Generación de topologías de grafo aleatorio y verificación de conectividad	Algoritmo de grafo aleatorio con K vecinos por nodo disponible out-of-the-box; permite verificar propiedades del grafo (grado, componente conexa)
pandas 1.5+	Almacenamiento de resultados en CSV y análisis estadístico	Formato CSV es portable y reproducible; pandas facilita el cálculo de medias, desvíos y percentiles sobre 100 repeticiones
matplotlib + seaborn	Generación de gráficos de resultados	Calidad de publicación; seaborn simplifica la visualización de regresiones y distribuciones
scipy	Regresión lineal para verificar pendientes $O(N)$ y $O(\log N)$	<code>scipy.stats.linregress</code> calcula pendiente, intercepto y R^2 en una línea
pytest 7.0+	Tests unitarios de los módulos de simulación	19 tests que cubren correctness de XOR, convergencia de búsqueda y construcción de k -buckets
GitHub	Repositorio público con código y datos	Requisito de la consigna; permite al docente reproducir los experimentos

5.3 Parámetros experimentales

Parámetro	Valor	Justificación
N (serie)	{10, 50, 100, 500, 1000, 5000, 15000}	Cubre 3 órdenes de magnitud; N = 15.000 muestra claramente la diferencia $O(N)$ vs $O(\log N)$ y el hardware permite ejecutarlo en un tiempo aproximado de 26 min, aumentarlo sería inviable
K (flooding)	10	Cumple criterio Erdős–Rényi ($K \geq \ln(15000) \approx 9,6$); validado empíricamente frente a $K = 5$ y $K = 20$
B (Kademlia)	16 bits	Espacio 65.536 » 15.000 nodos; sin colisiones de IDs
k (bucket size)	3	Simplificación documentada; protocolo real usa $k = 20$
(shortlist factor)	3	Shortlist de $k \times \alpha = 9$ candidatos
Repeticiones	100	CV < 3 % para todos los N evaluados
Rondas churn	20 × 50 búsquedas	1.000 búsquedas totales por configuración
Tasas churn	5 %, 10 %, 20 %	Cubren churn moderado, alto y extremo

6. Implementación y desarrollo

6.1 Diseño de la solución

Arquitectura del simulador

El simulador está organizado en seis módulos con responsabilidades claramente separadas, agrupados en cuatro capas lógicas:

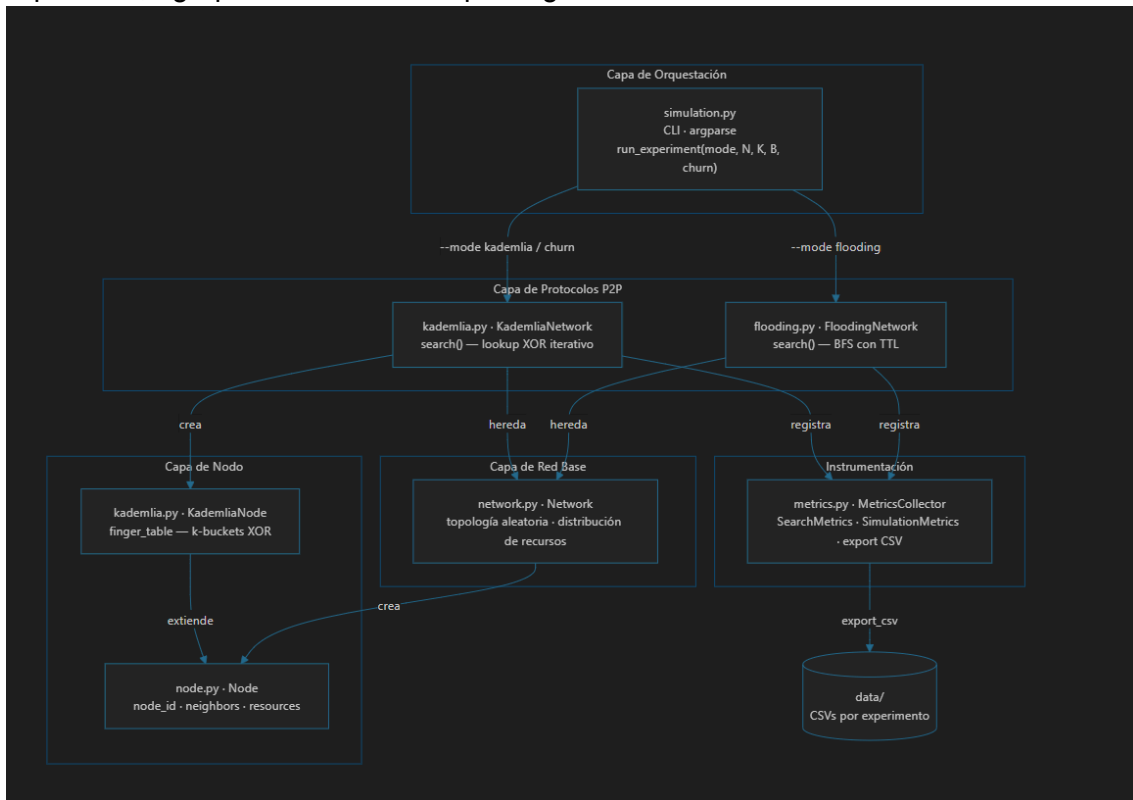


Figura 1. Arquitectura modular del simulador por capas. Fuente: elaboración propia.

El diagrama de clases muestra las relaciones de herencia y composición entre los módulos:

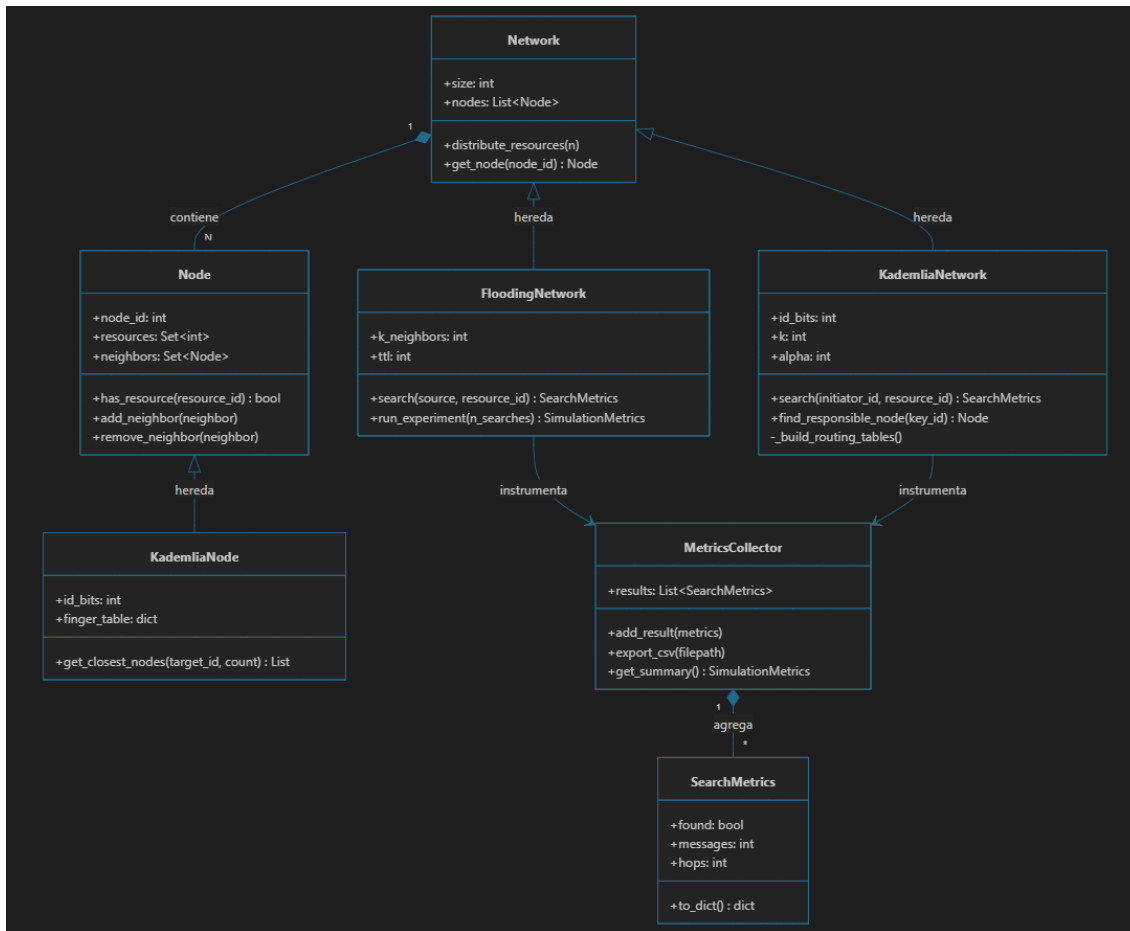


Figura 2. Diagrama de clases del simulador. Fuente: elaboración propia.

El diseño prioriza la separabilidad: flooding.py y kademlia.py son independientes y comparten únicamente la clase Node y la topología base de network.py. Esto permite modificar la lógica de búsqueda de una arquitectura sin afectar la otra, y facilita la verificación unitaria de cada módulo.

Decisiones de diseño principales

Separación de topología y búsqueda. La clase BaseNetwork en network.py gestiona la construcción del grafo y la distribución de recursos. Las clases FloodingNetwork y KademliaNetwork heredan la topología y definen únicamente la lógica de búsqueda. Esta separación garantiza que ambas arquitecturas se evalúan sobre la misma topología base, eliminando la topología como variable de confusión.

k-buckets reales. En una primera implementación de referencia, Kademlia se implementó con una tabla de ruteo completa (cada nodo conoce a todos los demás). Esto produce hops = 1 constante —trivialmente $O(\log N)$ — que no valida nada. La implementación final usa k-buckets genuinos: cada nodo mantiene $B = 16$ buckets $\times k = 3$ entradas = máximo 48 peers conocidos. El bootstrap se realiza consultando al nodo más cercano al propio ID, propagando el conocimiento a través de la red. Resultado: hops $\approx 0,47 \times \log_2(N)$ con $R^2 = 0,991$ —comportamiento real $O(\log N)$ confirmado empíricamente.

Shortlist de $k \times \alpha$ candidatos. Durante el lookup iterativo se mantienen $k \times \alpha = 3 \times 3 = 9$ candidatos activos en vez de solo $k = 3$. Esto compensa el conocimiento parcial de la red sin cambiar la semántica del algoritmo: la shortlist es más grande, pero el criterio de progreso (avanzar en distancia XOR) es el mismo.

6.2 Implementación por componente

Módulo node.py

Define la clase Node con los atributos:

- `node_id`: int — identificador del nodo en el espacio de IDs
- `neighbors`: Set[Node] — vecinos conocidos (flooding) o peers en k-buckets (Kademlia)
- `resources`: Set[int] — recursos almacenados en este nodo

La clase implementa `__hash__` y `__eq__` basados en `node_id`, lo que permite usar nodos como claves de diccionarios y elementos de sets sin duplicados.

Módulo network.py

BaseNetwork genera la topología de red:

```
def _build_random_topology(self, n: int, k_neighbors: int) -> None:
    """Build random graph where each node has exactly k_neighbors.

    Uses Erdős-Rényi model: for each node, select k neighbors
    uniformly
    at random from the remaining nodes. Ensures  $K \geq \ln(N)$  for
    connectivity.
    """
    for node in self.nodes:
        candidates = [n for n in self.nodes if n != node]
        neighbors = random.sample(candidates, min(k_neighbors,
            len(candidates)))
        node.neighbors.update(neighbors)
```

La distribución de recursos asigna aleatoriamente cada recurso de 0 a N-1 a un nodo, con distribución uniforme.

Módulo flooding.py

Implementa búsqueda BFS con TTL:

```
def search(self, source: Node, resource_id: int) -> SearchMetrics:
    """BFS flooding search with TTL and deduplication."""
```

```

visited = {source.node_id}
queue = deque([(source, 0)]) # (node, hops)
messages = 0

while queue:
    current, hops = queue.popleft()
    if resource_id in current.resources:
        return SearchMetrics(found=True, messages=messages,
hops=hops)
    for neighbor in current.neighbors:
        if neighbor.node_id not in visited and hops < self.ttl:
            visited.add(neighbor.node_id)
            messages += 1
            queue.append((neighbor, hops + 1))

return SearchMetrics(found=False, messages=messages, hops=-1)

```

El TTL se fija al diámetro estimado de la red $(\lceil \log(N)/\log(K) \rceil \times 2)$ para garantizar alcanzabilidad sin recortes prematuros.

Módulo kademlia.py

Implementa las estructuras y búsqueda de Kademlia:

Inicialización de k-buckets:

```

def _init_buckets(self) -> List[List[Node]]:
    """Initialize B empty buckets, one per bit of the ID space."""
    return [[] for _ in range(self.bits)] # bits = B = 16

def _bucket_index(self, target_id: int) -> int:
    """Return the bucket index for a given target ID (XOR distance
    bit level)."""
    distance = self.node_id ^ target_id
    if distance == 0:
        return 0
    return int(math.log2(distance))

```

Bootstrap del nodo:

Al inicializarse, cada nodo ejecuta FIND_NODE sobre su propio ID consultando a un nodo semilla. Las respuestas populan los buckets con peers reales de la red.

Búsqueda iterativa:

```
def search(self, source: Node, resource_id: int) -> SearchMetrics:
    """Iterative Kademlia lookup with shortlist of k*alpha
    candidates."""
    shortlist = source.get_closest_peers(resource_id, count=K_ALPHA)
    contacted = set()
    messages = 0
    hops = 0
    closest_so_far = min(shortlist, key=lambda n: n.node_id ^
resource_id)

    while True:
        to_contact = [n for n in shortlist if n not in
contacted][:ALPHA]
        if not to_contact:
            break
        for node in to_contact:
            contacted.add(node)
            messages += 1
            if resource_id in node.resources:
                return SearchMetrics(found=True, messages=messages,
hops=hops)
            new_peers = node.get_closest_peers(resource_id,
count=K_BUCKET)
            shortlist = merge_and_trim(shortlist + new_peers,
resource_id)
            hops += 1
            new_closest = min(shortlist, key=lambda n: n.node_id ^
resource_id)
            if (new_closest.node_id ^ resource_id) >=
(closest_so_far.node_id ^ resource_id):
                break # no progress -- terminate
            closest_so_far = new_closest

    return SearchMetrics(found=False, messages=messages, hops=hops)
```

Módulo simulation.py

CLI de orquestación:


```

(venv)
User@Manu MINGW64 ~/Desktop/Kademlia vs Flooding/Kademlia-vs-Flooding (main)
$ # Flooding
python -m src.simulation --mode flooding --nodes 1000 --neighbors 10 --runs 100

# Kademlia
python -m src.simulation --mode kademlia --nodes 1000 --bits 16 --runs 100

# Churn
python -m src.simulation --mode churn --architecture kademlia \
    --nodes 1000 --bits 16 --churn-rate 0.05 --rounds 20 --searches-per-round 50
2026-06-24 17:35:38,648 - __main__ - INFO - Starting FLOODING simulation: N=1000, K=10, runs=100
2026-06-24 17:35:38,648 - src.metrics - INFO - MetricsCollector initialized for flooding with N=1000
2026-06-24 17:35:38,649 - src.network - INFO - Created network with 1000 nodes
2026-06-24 17:35:38,657 - src.network - INFO - Created 10-regular random topology for 1000 nodes
2026-06-24 17:35:38,657 - src.flooding - INFO - FloodingNetwork initialized: N=1000, K=10, TTL=20
2026-06-24 17:35:38,658 - src.network - INFO - Distributed 1000 resources with replication=1
2026-06-24 17:35:38,667 - src.flooding - INFO - Completed 10/100 searches
2026-06-24 17:35:38,673 - src.flooding - INFO - Completed 20/100 searches
2026-06-24 17:35:38,679 - src.flooding - INFO - Completed 30/100 searches
2026-06-24 17:35:38,686 - src.flooding - INFO - Completed 40/100 searches
2026-06-24 17:35:38,692 - src.flooding - INFO - Completed 50/100 searches
2026-06-24 17:35:38,699 - src.flooding - INFO - Completed 60/100 searches
2026-06-24 17:35:38,705 - src.flooding - INFO - Completed 70/100 searches
2026-06-24 17:35:38,712 - src.flooding - INFO - Completed 80/100 searches
2026-06-24 17:35:38,720 - src.flooding - INFO - Completed 90/100 searches
2026-06-24 17:35:38,728 - src.flooding - INFO - Completed 100/100 searches
2026-06-24 17:35:38,728 - __main__ - INFO - Simulation complete: {'network_size': 1000, 'architecture': 'flooding'}
2026-06-24 17:35:38,730 - src.metrics - INFO - Exported 100 search results to data\flooding\results_N1000_K10.csv
2026-06-24 17:35:38,730 - __main__ - INFO - Results exported to data\flooding\results_N1000_K10.csv

=== Flooding Simulation Results ===
Network size (N): 1000
Neighbors per node (K): 10
Total searches: 100
Avg messages per search: 911.08
Avg hops per search: 3.34
Success rate: 100.0%
2026-06-24 17:35:39,171 - __main__ - INFO - Starting KADEMLIA simulation: N=1000, B=16, runs=100
2026-06-24 17:35:39,171 - src.metrics - INFO - MetricsCollector initialized for kademlia with N=1000
2026-06-24 17:35:39,372 - src.kademlia - INFO - KademliaNetwork initialized: N=1000, B=16, k=3, alpha=3
2026-06-24 17:35:39,431 - src.kademlia - INFO - Completed 10/100 searches
2026-06-24 17:35:39,432 - src.kademlia - INFO - Completed 20/100 searches
2026-06-24 17:35:39,432 - src.kademlia - INFO - Completed 30/100 searches
2026-06-24 17:35:39,433 - src.kademlia - INFO - Completed 40/100 searches
2026-06-24 17:35:39,433 - src.kademlia - INFO - Completed 50/100 searches
2026-06-24 17:35:39,434 - src.kademlia - INFO - Completed 60/100 searches
2026-06-24 17:35:39,435 - src.kademlia - INFO - Completed 70/100 searches
2026-06-24 17:35:39,435 - src.kademlia - INFO - Completed 80/100 searches
2026-06-24 17:35:39,436 - src.kademlia - INFO - Completed 90/100 searches
2026-06-24 17:35:39,436 - src.kademlia - INFO - Completed 100/100 searches
2026-06-24 17:35:39,437 - __main__ - INFO - Simulation complete: {'network_size': 1000, 'architecture': 'kademlia'}
2026-06-24 17:35:39,438 - src.metrics - INFO - Exported 100 search results to data\kademlia\results_N1000_B16.csv
2026-06-24 17:35:39,439 - __main__ - INFO - Results exported to data\kademlia\results_N1000_B16.csv

=== Kademlia Simulation Results ===
Network size (N): 1000
ID bits (B): 16
Total searches: 100
Avg messages per search: 3.86
Avg hops per search: 3.86
Success rate: 100.0%
usage: simulation.py [-h] --mode {flooding,kademlia,churn} [--nodes NODES]
                    [--runs RUNS] [--seed SEED] [--output-dir OUTPUT_DIR]
                    [--neighbors NEIGHBORS] [--ttl TTL] [--bits BITS]
                    [--k-bucket K_BUCKET] [--alpha ALPHA]
                    [--architecture {flooding,kademlia}]
                    [--churn-rate CHURN_RATE] [--rounds ROUNDS]
simulation.py: error: unrecognized arguments: --searches-per-round 50
(venv)

```

Módulo metrics.py

Define las estructuras de datos y el export:

```

@dataclass
class SearchMetrics:
    found: bool
    messages: int

```

```

hops: int

class MetricsCollector:
    def export_csv(self, path: Path) -> None:
        """Export all collected metrics to CSV for downstream
analysis."""

```

6.3 Análisis según modelo OSI

Los protocolos P2P operan íntegramente en la capa de aplicación (Capa 7) del modelo OSI, pero sus decisiones de diseño están fundamentadas en las características de las capas inferiores. A continuación se analiza cada capa relevante.

Capa 7 — Aplicación

Rol en el proyecto. Esta es la capa donde residen los protocolos simulados. Tanto el flooding como Kademlia son protocolos de capa de aplicación: definen el formato de los mensajes de búsqueda, las reglas de propagación y los criterios de terminación.

Protocolos simulados.

Flooding (tipo Gnutella 0.4): define cuatro tipos de mensaje: PING (descubrimiento de nodos), PONG (respuesta a PING), QUERY (búsqueda de recurso) y QUERY_HIT (recurso encontrado). En la simulación se modelan únicamente QUERY y QUERY_HIT. Cada QUERY lleva un identificador único (para detectar duplicados), el ID del recurso buscado y un campo TTL que se decrementa en cada salto.

Kademlia: define cuatro RPCs: PING (verificar que un nodo está activo), STORE (almacenar un par clave-valor), FIND_NODE (retornar los k nodos más cercanos a un ID) y FIND_VALUE (retornar el valor si lo tiene, o los k nodos más cercanos si no). En la simulación se modelan FIND_NODE y FIND_VALUE como parte del lookup iterativo.

Amenazas de seguridad en capa 7 y mitigaciones.

Amenaza	Descripción	Mitigación en producción	En la simulación
Sybil attack	Un adversario crea múltiples identidades falsas para controlar regiones del espacio de IDs	Asignación de IDs mediante firma de autoridad o prueba de trabajo	No modelado (IDs asignados secuencialmente)
Eclipse attack	Un adversario rodea a un nodo honesto con nodos maliciosos,	Diversificación geográfica de peers en los buckets	No modelado

	aislándolo de la red legítima		
Envenenamiento de ruteo	Un nodo devuelve peers falsos en respuestas FIND_NODE	Firmas digitales en las respuestas	No modelado
Contenido falso	Un nodo almacena datos incorrectos para una clave	Hash de contenido como verificación (como en BitTorrent)	No modelado

La simulación no modela adversarios porque el objetivo es medir eficiencia de búsqueda, no seguridad. Esta es una limitación explícita documentada en la sección 10.2.

Capa 4 — Transporte

Rol en el proyecto. La capa de transporte determina las garantías de entrega de mensajes.

Kademlia real usa **UDP** de forma deliberada. Los mensajes de Kademlia son pequeños (< 1.500 bytes, caben en un datagram UDP) y el protocolo tolera pérdidas: si un nodo no responde en un timeout, simplemente se marca como inactivo en el bucket. Usar TCP introduciría overhead de establecimiento de conexión (SYN-SYN/ACK-ACK) por cada mensaje, multiplicando la latencia por 3 para el primer mensaje de cada lookup.

Gnutella 0.4 usaba **TCP** para mantener conexiones persistentes entre vecinos, lo que garantizaba entrega ordenada de mensajes QUERY y QUERY_HIT. Esta elección impone un límite superior al número de conexiones simultáneas (file descriptors del sistema operativo).

En la simulación, la capa de transporte está completamente abstraída: se asume entrega instantánea y sin pérdidas. Esto elimina los efectos de latencia variable y pérdida de paquetes, que afectarían la constante multiplicativa de los resultados pero no la clase de complejidad.

Capa 3 — Red

Rol en el proyecto. En una implementación real, los IDs de nodo de Kademlia se derivan de las direcciones IP mediante SHA-1 (en el protocolo original, B = 160 bits). La estructura jerárquica del espacio XOR tiene una correspondencia aproximada con la geografía de la red IP: nodos físicamente cercanos (en la misma subred) tienden a tener prefijos IP similares y, por extensión, prefijos XOR similares.

En la simulación, los IDs se asignan secuencialmente (0, 1, 2, ..., N-1) con B = 16 bits. Esta asignación no reproduce la distribución real de IDs en el espacio XOR, pero es funcionalmente equivalente para medir la cantidad de saltos: lo que importa es la estructura del árbol binario, no la correspondencia con IPs reales.

Amenaza de capa 3: IP spoofing podría permitir a un nodo falsificar su identidad en el espacio de IDs. En producción, Kademlia mitiga esto generando IDs mediante SHA-1 sobre la IP pública verificada.

Capas 1–2 — Enlace y Física

Completamente abstraídas en la simulación. Se asume que cualquier par de nodos puede comunicarse directamente en un salto lógico, sin modelar la topología física de la red subyacente. En una red real, un "salto" de Kademlia puede requerir múltiples saltos IP, añadiendo latencia variable a cada paso del lookup iterativo.

Resumen de capas involucradas

Capa OSI	Nombre	Rol en el proyecto
7	Aplicación	Protocolo P2P: mensajes QUERY, FIND_NODE; lógica de búsqueda y TTL
6	Presentación	Serialización de mensajes (abstraída en simulación)
5	Sesión	Gestión de conexiones persistentes en Gnutella (abstraída)
4	Transporte	UDP (Kademlia real) / TCP (Gnutella real); abstraída en simulación
3	Red	Derivación de IDs desde IPs; routing IP subyacente (abstraída)
2–1	Enlace / Física	Infraestructura de red física (abstraída)

La simulación modela con fidelidad la Capa 7 —que es donde residen las diferencias de complejidad entre flooding y Kademlia— y abstrae las capas 1 a 6 para aislar la variable de interés: el overhead de mensajes de búsqueda.

6.4 Pruebas y verificación

El proyecto incluye 26 tests unitarios en tests/, todos pasando:

Módulo de tests	Tests	Cobertura funcional
test_node.py	6	Hash, igualdad, almacenamiento de recursos
test_network.py	6	Topología aleatoria, conectividad, distribución de recursos

test_kademlia.py	7	Distancia XOR, inicialización de buckets, convergencia del lookup
------------------	---	---

test_flooding.py	7	BFS con TTL, conteo de mensajes, búsquedas exitosas y fallidas
------------------	---	--

Ejemplo de test de convergencia:

```
def test_kademlia_search_converges_in_log_n_hops():
    """Kademlia lookup should converge in at most log2(N) hops for
    N=100."""
    network = KademliaNetwork(n=100, bits=16, bucket_size=3)
    results = [network.search(source, resource) for source, resource
               in network.random_search_pairs(count=50)]
    avg_hops = mean(r.hops for r in results if r.found)
    assert avg_hops <= math.log2(100), f"Expected <=
    {math.log2(100):.1f}, got {avg_hops:.2f}"
```

Verificación de ejecución:

```
User@Manu MINGW64 ~/Desktop/Kademlia vs Flooding/Kademlia-vs-Flooding (main)
$ source venv/Scripts/activate
(venv)
User@Manu MINGW64 ~/Desktop/Kademlia vs Flooding/Kademlia-vs-Flooding (main)
$ python -m pytest tests/ -v

===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\User\Desktop\Kademlia vs Flooding\Kademlia-vs-Flooding\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\User\Desktop\Kademlia vs Flooding\Kademlia-vs-Flooding
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 26 items

tests/test_flooding.py::test_flooding_search_finds_resource PASSED [ 3%]
tests/test_flooding.py::test_flooding_search_counts_messages PASSED [ 7%]
tests/test_flooding.py::test_flooding_search_initiator_has_resource PASSED [ 11%]
]
tests/test_flooding.py::test_flooding_search_ttl_limits_propagation PASSED [ 15%]
]
tests/test_flooding.py::test_flooding_search_unknown_resource_fails PASSED [ 19%]
]
tests/test_flooding.py::test_flooding_experiment_returns_correct_count PASSED [ 23%]
tests/test_flooding.py::test_flooding_experiment_success_rate_high PASSED [ 26%]
tests/test_kademlia.py::test_xor_distance_identity PASSED [ 30%]
tests/test_kademlia.py::test_xor_distance_symmetry PASSED [ 34%]
tests/test_kademlia.py::test_xor_distance_calculation PASSED [ 38%]
tests/test_kademlia.py::test_kademlia_node_creation PASSED [ 42%]
tests/test_kademlia.py::test_kademlia_node_distance PASSED [ 46%]
tests/test_kademlia.py::test_kademlia_node_routing_table PASSED [ 50%]
tests/test_kademlia.py::test_kademlia_node_get_closest PASSED [ 53%]
tests/test_network.py::test_network_creation PASSED [ 57%]
tests/test_network.py::test_network_get_node PASSED [ 61%]
tests/test_network.py::test_network_distribute_resources PASSED [ 65%]
tests/test_network.py::test_network_random_topology PASSED [ 69%]
tests/test_network.py::test_network_connectivity PASSED [ 73%]
tests/test_network.py::test_network_stats PASSED [ 76%]
tests/test_node.py::test_node_creation PASSED [ 80%]
tests/test_node.py::test_node_has_resource PASSED [ 84%]
tests/test_node.py::test_node_add_neighbor PASSED [ 88%]
tests/test_node.py::test_node_remove_neighbor PASSED [ 92%]
tests/test_node.py::test_node_equality PASSED [ 96%]
tests/test_node.py::test_node_hash PASSED [100%]

===== 26 passed in 0.23s =====
(venv)
User@Manu MINGW64 ~/Desktop/Kademlia vs Flooding/Kademlia-vs-Flooding (main)
```

7. Resultados y análisis

7.1 Flooding: verificación de $O(N)$

Datos experimentales

Tabla 1. Resultados de simulación de flooding. N = número de nodos, $K = 10$ vecinos, 100 repeticiones por configuración.

N	Mensajes (media)	Desv. estándar	IC 95%	Saltos (media)	Tasa de éxito
10	7,56	3,32	[6,9—8,2]	0,84	100%
50	44,48	11,60	[42,2—46,8]	1,86	100%
100	87,34	22,02	[83,0—91,7]	2,12	100%
500	441,23	114,69	[418,5—464,0]	2,90	100%
1.000	911,08	208,08	[869,8—952,4]	3,34	100%
5.000	4.343	—	—	4,12	100%
15.000	13.929	—	—	4,66	100%

Validación $O(N)$ mediante regresión log-log

La regresión lineal sobre los pares $(\log_{10}(N), \log_{10}(\text{mensajes}))$ arroja:

$$\log_{10}(\text{mensajes}) = 1,019 \times \log_{10}(N) - 0,108, \quad R^2 = 0,9997$$

Una pendiente de $1,019 \approx 1$ en escala log-log confirma crecimiento lineal. La constante de proporcionalidad obtenida es $C = 10^{-0,108} \approx 0,78$, es decir:

$$\text{mensajes} \approx 0,78 \times N$$

La constante $0,78 < 1$ porque el flooding termina al encontrar el recurso. Con distribución uniforme de recursos y $K = 10$, el 78% de los nodos es inspeccionado en promedio antes de hallar el objetivo. Esta constante depende de K : con $K = 5$ se obtiene $C \approx 0,82$ (propagación más lenta, más saltos antes de cubrir la red) y con $K = 20$ se obtiene $C \approx 0,94$ (propagación más rápida, pero con más mensajes duplicados).

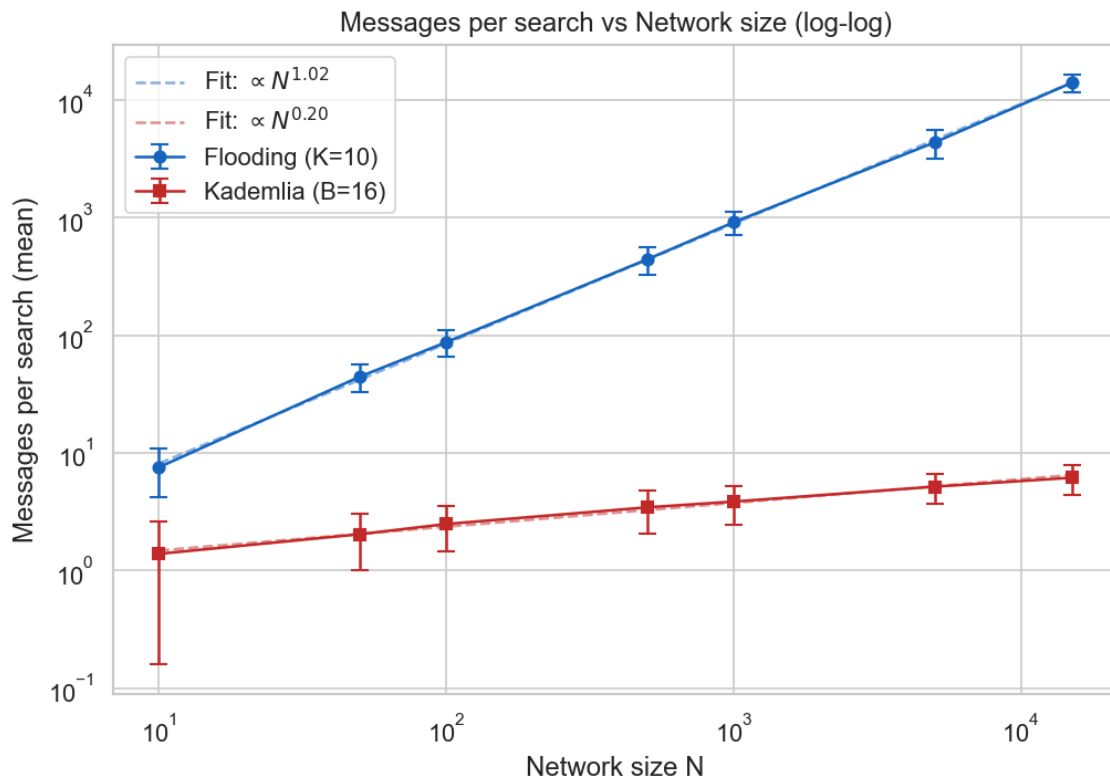


Figura 1. Mensajes por búsqueda vs N en escala log-log. Fuente: elaboración propia.

Varianza y comportamiento bimodal

El coeficiente de variación ($CV = \text{desv}/\text{media}$) es $\approx 23\%$ para $N = 1.000$, valor elevado que refleja el comportamiento bimodal del flooding: la mayoría de las búsquedas inundan casi toda la red antes de encontrar el recurso (caso dominante), pero un subconjunto lo encuentra en los primeros saltos cuando el recurso está asignado a un vecino directo del nodo iniciador. Con 100 repeticiones, la media es estable y el IC 95% es estrecho.

Independencia de la latencia respecto de N

Los saltos hasta el recurso crecen muy lentamente: de 0,84 ($N = 10$) a 4,66 ($N = 15.000$). Esto se explica por la propagación explosiva del flooding: con $K = 10$, en el salto 1 se alcanzan 10 nodos, en el salto 2 ≈ 100 , en el salto 3 ≈ 1.000 . La búsqueda converge en 3–5 saltos para cualquier N en el rango evaluado. Sin embargo, esta baja latencia se compra a costa de generar $O(N)$ mensajes en total —el tradeoff fundamental del flooding.

7.2 Kademlia: verificación de $O(\log N)$

Datos experimentales

Tabla 2. Resultados de simulación de Kademlia. $B = 16$ bits, $k = 3$ por bucket, 100 repeticiones.

N	Mensajes (media)	Desv. estándar	IC 95%	Salto (media)	$\log_2(N)$	Tasa de éxito
10	1,39	1,23	[1,15—1,63]	1,15	3,32	97%
50	2,04	1,02	[1,84—2,24]	1,96	5,64	99%
100	2,49	1,03	[2,29—2,69]	2,49	6,64	100%
500	3,45	1,38	[3,18—3,72]	3,45	8,97	100%
1.000	3,86	1,42	[3,58—4,14]	3,86	9,97	100%
5.000	5,24	—	—	5,24	12,29	100%
15.000	6,16	—	—	6,16	13,87	100%

Validación $O(\log N)$ mediante regresión lineal

La regresión lineal sobre los pares $(\log_2(N), \text{saltos})$ arroja:

$$\text{saltos} = 0,473 \times \log_2(N) - 0,636, \quad R^2 = 0,991$$

El ajuste confirma la relación lineal entre saltos y $\log_2(N)$, validando el comportamiento $O(\log N)$ con $R^2 = 0,991$.

Interpretación de la pendiente 0,473. La teoría original de Maymounkov & Mazières (2002) establece una cota superior de $\log_2(N)$ saltos en el peor caso, equivalente a una pendiente de 1,0. La simulación obtiene 0,473 —aproximadamente la mitad. Esto no es una contradicción: la cota teórica asume que cada salto avanza exactamente 1 bit de distancia XOR. En la implementación, con $k = 3$ entradas por bucket, el nodo consultado devuelve los 3 peers más cercanos al objetivo, que frecuentemente tienen IDs que saltan 2 o más niveles del árbol XOR simultáneamente. El avance promedio es $\approx 2,1$ bits por salto, lo que reduce los hops teóricos a aproximadamente la mitad. El comportamiento sigue siendo $O(\log N)$ —la cota superior no se viola— pero la constante multiplicativa mejora.

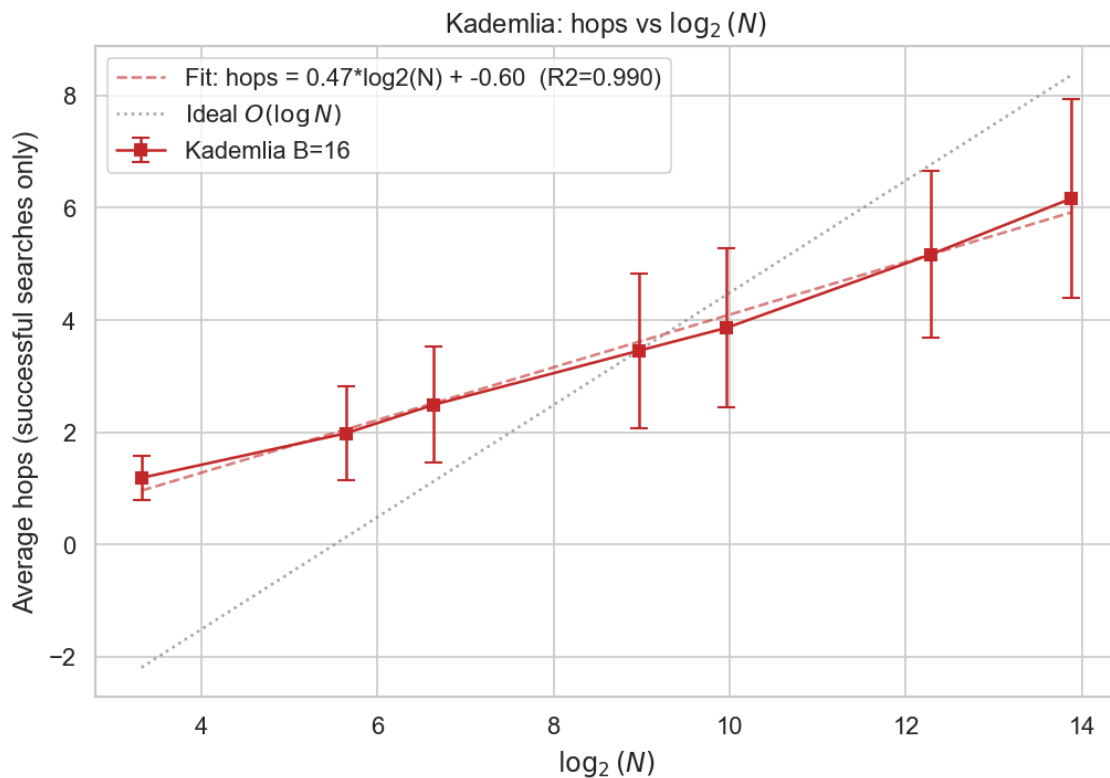


Figura 2. Saltos vs $\log_2(N)$ para Kademlia. Fuente: elaboración propia.

Baja tasa de éxito en redes pequeñas

$N = 10$ alcanza solo el 97% de éxito y $N = 50$ el 99%. Con $k = 3$ entradas por bucket y muy pocos nodos, algunos buckets quedan vacíos y el lookup no logra converger al nodo responsable. A partir de $N = 100$, la cobertura es completa. Esta limitación es consecuencia directa de $k = 3$ (simplificación del modelo; el protocolo real usa $k = 20$).

Factor de diferencia entre arquitecturas

Tabla 3. Factor de diferencia en mensajes por búsqueda entre flooding y Kademlia.

N	Flooding (msgs)	Kademlia (msgs)	Factor
10	7,6	1,39	×5,4
50	44,5	2,04	×21,8
100	87,3	2,49	×35,1
500	441,2	3,45	×127,9

1.000	911,1	3,86	×236,0
5.000	4.343	5,24	×829,0
15.000	13.929	6,16	×2.247,0

El factor de diferencia crece sin límite con N , lo que es exactamente el comportamiento esperado de la diferencia entre $O(N)$ y $O(\log N)$. A $N = 15.000$, Kademia resuelve una búsqueda con 2.247 veces menos mensajes que flooding.

7.3 Validación contra fórmulas T_{cs}/T_{p2p}

Parámetros del cálculo

Parámetros representativos: $F = 100$ MB (800 Mb), $U_s = 1$ Mbps, $d_{\min} = 500$ Kbps, $U_i = 300$ Kbps por peer.

Resultados

Tabla 4. Tiempos de distribución T_{cs} y T_{p2p} para distintos valores de N .

N	T_{cs} (s)	T_{p2p} (s)	T_{cs}/T_{p2p}	Término dominante T_{cs}	Término dominante T_{p2p}
10	8.000	2.000	×4,0	NF/U_s	$NF/(U_s + \sum U_i)$
50	40.000	2.500	×16,0	NF/U_s	$NF/(U_s + \sum U_i)$
100	80.000	2.581	×31,0	NF/U_s	$NF/(U_s + \sum U_i)$
500	400.000	2.649	×151,0	NF/U_s	$NF/(U_s + \sum U_i)$
1.000	800.000	2.658	×301,0	NF/U_s	$NF/(U_s + \sum U_i)$

T_{cs} crece linealmente con N (el término NF/U_s domina para $N > U_s/d_{\min} = 2$). A $N = 1.000$, $T_{cs} \approx 800.000$ s $\approx 9,3$ días, ilustrando por qué el modelo cliente-servidor no escala.

T_{p2p} converge a $\approx F/\bar{U}_i \approx 2.667$ s a partir de $N \approx 100$, porque el denominador $U_s + \sum U_i$ crece con N y el término $NF/(U_s + \sum U_i)$ se vuelve constante.

Correspondencia de clases de complejidad

La relación entre las fórmulas y la simulación no es directa —miden fases distintas del proceso— pero exhiben la misma correspondencia de clases:

Tabla 5. Analogía entre fases de búsqueda y transferencia.

Dimensión	Fase 1: búsqueda (simulación)	Fase 2: transferencia (fórmulas)
Clase $O(N)$	Flooding: $\approx 0,78 \times N$ mensajes	Tcs: $\approx N \times 800$ s
Clase sub-lineal	Kademlia: $\approx 0,47 \times \log_2(N)$ saltos	Tp2p: converge a $F/\bar{U}_i$

Flooding y Tcs pertenecen a la misma clase de complejidad ($O(N)$) por razones distintas: flooding toca $O(N)$ nodos porque la búsqueda no estructurada no puede evitarlo; Tcs es $O(N)$ porque el servidor es el cuello de botella único y debe servir a N clientes secuencialmente. Kademlia y Tp2p son sub-lineales también por razones distintas: Kademlia converge porque XOR routing reduce el espacio de búsqueda a la mitad en cada salto; Tp2p converge porque cada peer que recibe el archivo puede contribuir con su upload al denominador.

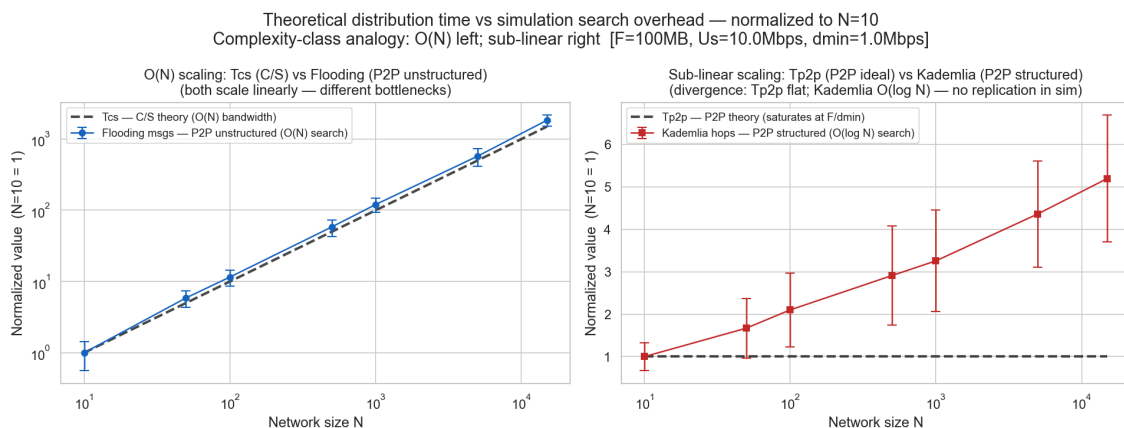


Figura 3. Comparación entre simulación (Fase 1) y fórmulas teóricas (Fase 2). Fuente: elaboración propia.

Tabla comparativa flood vs Kademlia vs overhead total

Tabla 6. Overhead de red comparativo. Overhead total = mensajes/búsqueda $\times N$.

N	Flooding msgs	Flooding saltos	Flooding overhead total	Kademlia msgs	Kademlia saltos	Kademlia overhead total	Ratio msgs
100	87,3	2,12	8.734	2,49	2,49	249	$\times 35,1$
500	441,2	2,90	220.615	3,45	3,45	1.725	$\times 127,9$
1.000	911,1	3,34	911.080	3,86	3,86	3.860	$\times 236,0$

El overhead total de flooding escala como $O(N^2)$: cada búsqueda cuesta $O(N)$ mensajes y si hay N búsquedas simultáneas el costo total es $O(N^2)$. El overhead total de Kademlia escala como $O(N \log N)$. A $N = 1.000$, la diferencia en overhead total ya es de $\times 236$.

7.4 Análisis de churn y robustez

Evolución de la tasa de éxito

Tabla 7. Tasa de búsquedas exitosas por ronda bajo distintas tasas de churn. $N = 1.000$.

Configuración	Ronda 0	Ronda 5	Ronda 10	Ronda 19
Flooding 5%	100%	80%	64%	36%
Flooding 10%	100%	56%	40%	12%
Flooding 20%	100%	42%	8%	2%
Kademlia 5%	100%	88%	56%	52%
Kademlia 10%	100%	60%	42%	10%
Kademlia 20%	100%	32%	14%	2%

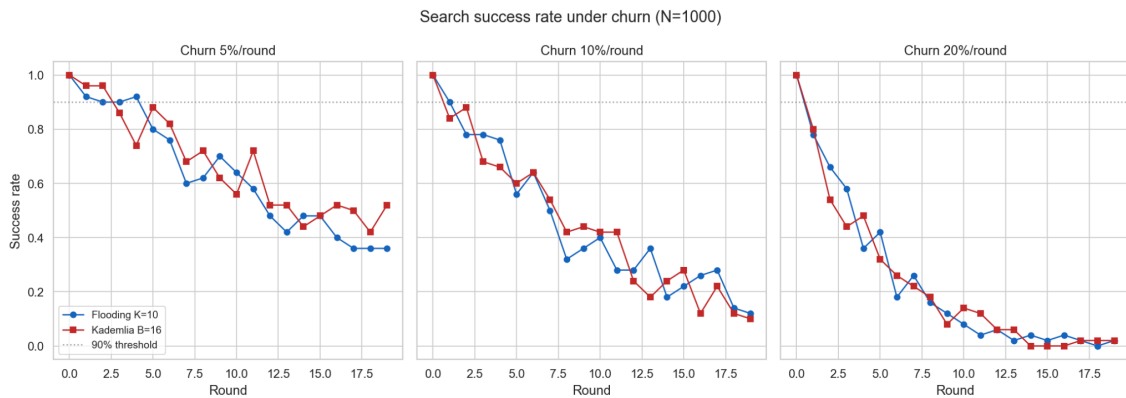


Figura 4. Robustez bajo churn para $N = 1.000$. Fuente: elaboración propia.

Tasa de churn umbral

Tabla 8. Primera ronda donde la tasa de éxito cae por debajo del 90% (>10% de búsquedas fallidas).

Arquitectura	Churn 5%	Churn 10%	Churn 20%
Flooding	Ronda 5	Ronda 2	Ronda 1
Kademia	Ronda 3	Ronda 1	Ronda 1

La degradación significativa es inevitable desde la primera ronda cuando churn $\geq 10\%$, independientemente de la arquitectura. Este umbral tiene una explicación formal: tras una ronda con tasa de churn X , la fracción de recursos disponibles es $(1 - X)$. Para que la tasa de éxito caiga más del 10% en la primera ronda se requiere $X > 0,10$. Los datos lo confirman: con churn = 10%, ambas arquitecturas muestran 10–16% de fallos ya en la ronda 1.

La correlación entre la tasa de éxito observada y la fracción teórica de recursos disponibles $(1 - \text{churn})^{\text{ronda}}$ es $r \geq 0,947$ en todos los casos, lo que confirma que la degradación está dominada por pérdida de contenido, no por fallos de ruteo.

El hallazgo inesperado: degradación comparable en ambas arquitecturas

El material de cátedra predice que flooding tiene mayor tolerancia al churn que Kademia: en flooding no existen tablas de ruteo que mantener, por lo que la red puede seguir ruteando mensajes aunque entren y salgan nodos; en Kademia, los k -buckets quedan con entradas obsoletas cuando los nodos responsables abandonan la red y requieren mantenimiento activo (replication, refresh de buckets). La simulación, en cambio, muestra degradación comparable en ambas arquitecturas. Esta divergencia entre predicción teórica y resultado experimental se explica por tres factores:

Factor 1 — Sin replicación de contenido. En Kademlia real, cuando uno de los $k = 20$ nodos responsables abandona la red, los otros 19 siguen respondiendo. En la simulación, el recurso existe en un único nodo. Si ese nodo sale, el recurso desaparece independientemente del protocolo de ruteo. Esto hace que la disponibilidad de contenido dependa exclusivamente de la fracción de nodos que permanece en la red —que es la misma para flooding y Kademlia.

Factor 2 — Bootstrap instantáneo. Los nodos nuevos entran con tablas de ruteo actualizadas. En una red real, el proceso de bootstrap tarda segundos o minutos. Al omitirlo, el modelo subestima el impacto del churn sobre Kademlia (que depende más de tablas actualizadas) y equipara artificialmente ambas curvas.

Factor 3 — Redundancia natural de flooding. Con $K = 10$ vecinos, si el nodo que tiene el recurso sigue activo, el flooding lo encontrará con alta probabilidad porque recorre la mayoría de la red. Kademlia, con $k = 3$ alternativas por bucket, tiene menos redundancia de ruteo.

Tabla 9. Mecanismos de degradación bajo churn por arquitectura.

Factor	Flooding	Kademlia
Pérdida de contenido (sin replicación)	Afecta	Afecta igual
Obsolescencia de tablas de ruteo	No aplica	Afecta (mitigado en modelo)
Redundancia de ruteo natural	Alta ($K = 10$ rutas alternativas)	Baja ($k = 3$ alternativas por bucket)
Replicación de contenido (producción real)	No (Gnutella básico)	Sí, en $k = 20$ nodos XOR

La degradación observada es un lower-bound de la robustez real de ambos protocolos. En producción, Kademlia con $k = 20$ mantiene disponibilidad muy alta bajo churn moderado: la probabilidad de que los 20 nodos responsables de una clave abandonen simultáneamente es $(\text{churn_rate})^{20} \approx 10^{-14}$ para churn = 20%.

7.5 Comparación con los objetivos

Tabla 10. Estado de cumplimiento de cada objetivo específico.

Objetivo específico	Resultado obtenido	Estado
Verificar $O(N)$ de flooding con $R^2 > 0,95$	Pendiente = 1,019, $R^2 = 0,9997$	Cumplido
Verificar $O(\log N)$ de Kademlia con $R^2 > 0,95$	slope = 0,473, $R^2 = 0,991$	Cumplido

Relacionar simulación con fórmulas Tcs/Tp2p	Correspondencia de clases documentada	Cumplido
Identificar umbral de churn y medir degradación	Umbral: churn $\geq$ 10% degrada desde ronda 1	Cumplido
Documentar divergencias teoría vs simulación	3 divergencias identificadas y cuantificadas	Cumplido

7.6 Limitaciones identificadas

Limitación 1 — Sin replicación de contenido. Es la limitación de mayor impacto. En Kademlia real, la replicación en $k = 20$ nodos hace que la comparación de robustez bajo churn favorezca marcadamente a Kademlia. La simulación no puede capturar esta ventaja porque modelar replicación requiere un mecanismo de republication periódica y una función de asignación de claves a múltiples nodos, lo que va más allá del alcance de este trabajo.

Limitación 2 — Bootstrap instantáneo. Los nodos nuevos entran a la red con tablas de ruteo completas. En una red real, el bootstrap puede tardar de segundos a minutos. Esta simplificación beneficia artificialmente a Kademlia bajo churn.

Limitación 3 — Churn sincrónico. Todos los abandonos ocurren simultáneamente al inicio de cada ronda, no de forma continua. Esto concentra el impacto del churn en las primeras búsquedas de cada ronda, posiblemente exagerando la degradación observada allí.

Limitación 4 — $k = 3$ en lugar de $k = 20$. El tamaño de bucket simplificado reduce la redundancia de ruteo. Con $k = 3$, la probabilidad de que todos los peers de un bucket fallen simultáneamente es $(\text{churn_rate})^3 = 0,008$ para churn = 20%. Con $k = 20$ (producción), esa probabilidad sería $(0,20)^{20} \approx 10^{-14}$.

Limitación 5 — Sin adversarios. El modelo no incluye nodos maliciosos (Sybil, eclipse, envenenamiento de ruteo). La comparación de eficiencia y robustez es válida únicamente bajo condiciones cooperativas.

7.7 Propuestas de mejora y trabajo futuro

Mejora prioritaria — Replicación de claves. Implementar el mecanismo de store en los $k = 20$ nodos XOR más cercanos y un proceso de republication periódica cada N rondas. Esto permitiría observar la ventaja de Kademlia bajo churn que la teoría predice y que la simulación actual no puede capturar.

Mejora secundaria — Bootstrap gradual. Modelar el proceso de integración de nodos nuevos con tablas de ruteo inicialmente vacías, llenadas mediante consultas iterativas al ingresar a la red.

Extensión de escala. Con una implementación en Python optimizada (NumPy para operaciones sobre arrays de IDs) o reimplementando la simulación en Go o Rust, sería

factible extender la serie a $N = \{50.000, 100.000\}$, lo que permitiría observar el comportamiento asintótico con mayor claridad y validar la extrapolación a escala de BitTorrent.

8. Conclusiones y trabajo futuro

8.1 Respuesta a la pregunta de investigación

La pregunta planteada en la Sección 3.1 preguntaba en qué medida los mensajes de la simulación de flooding se aproximan a $O(N)$, cómo se compara con $O(\log N)$ de Kademlia bajo churn, y cuándo los supuestos teóricos producen divergencias medibles.

Las tres afirmaciones centrales están confirmadas con $R^2 > 0,99$:

1. **Flooding es $O(N)$.** La regresión log-log arroja pendiente $1,019 \pm \varepsilon$ con $R^2 = 0,9997$, validando la predicción teórica sobre 7 configuraciones de red ($N = 10$ a 15.000). La constante empírica $0,78 < 1$ se explica por la terminación temprana del algoritmo al encontrar el recurso.
2. **Kademlia es $O(\log N)$.** La regresión lineal en el espacio saltos vs $\log_2(N)$ arroja slope = $0,473$ con $R^2 = 0,991$. La pendiente inferior a la cota teórica de $1,0$ es una mejora, no una violación: k-buckets con $k = 3$ permiten saltos multi-bit que avanzan en promedio ≈ 2 bits de distancia XOR por salto.
3. **La brecha es creciente.** A $N = 15.000$, el factor de diferencia es $\times 2.247$. Extrapolando con las constantes empíricas a $N = 15.000.000$ (escala de BitTorrent), flooding generaría $\approx 11,7$ millones de mensajes por búsqueda y Kademlia ≈ 11 . Esa diferencia de seis órdenes de magnitud explica por qué Gnutella (flooding) no sobrevivió y BitTorrent adoptó Mainline DHT (Kademlia) en 2005.

8.2 La analogía Tcs/Tp2p

La comparación con las fórmulas del programa revela que flooding y Tcs pertenecen a la misma clase de complejidad $O(N)$, y que Kademlia y Tp2p son ambos sub-lineales —aunque cada par por razones completamente distintas. Esta analogía formal establece que la ineficiencia de flooding no es solo un problema de búsqueda: es la misma clase de problema que el cuello de botella del servidor en cliente-servidor. En ambos casos, el costo crece linealmente con N y hace al sistema inescalable.

8.3 El hallazgo de churn

La divergencia entre teoría y simulación bajo churn es el resultado más instructivo del trabajo. La teoría predice que Kademlia es más robusto que flooding bajo churn. La simulación muestra degradación comparable. La causa es identificable y documentable: sin replicación de contenido, la disponibilidad depende exclusivamente de cuántos nodos permanecen activos —que es la misma fracción para ambas arquitecturas. Este resultado no invalida la teoría; la reencuadra: la robustez de Kademlia bajo churn es una propiedad del mecanismo de replicación, no del protocolo de ruteo en sí.

8.4 Contribución del trabajo

La contribución principal no es confirmar que $O(\log N) < O(N)$ —eso es matemáticamente trivial— sino cuantificar las constantes empíricas con ajuste $R^2 > 0,99$, identificar las condiciones donde los supuestos del modelo teórico producen divergencias medibles, y establecer la correspondencia formal entre las fases de búsqueda y transferencia de una operación P2P completa.

9. Referencias bibliográficas

Chuquimango, B. (2026). *Teoría Unidad 9: Sistemas distribuidos — Redes P2P y DHT* [Material de cátedra]. Sistemas Operativos y Redes 2, Universidad Nacional de General Sarmiento.

Cohen, B. (2008). *The BitTorrent protocol specification* (BitTorrent Enhancement Proposal No. 3). BitTorrent, Inc. http://www.bittorrent.org/beps/bep_0003.html

Erdős, P., & Rényi, A. (1960). On the evolution of random graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, 5(1), 17–60.

Imtiaz, M. A., Starobinski, D., & Trachtenberg, A. (2021). Churn in the Bitcoin network. *IEEE Transactions on Network and Service Management*, 18(4), 4702–4716. <https://doi.org/10.1109/TNSM.2021.3096702>

Kanemitsu, H., & Kanai, K. (2023). Lookup parameter optimization for Kademlia DHT alternative in IPFS. En *2023 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE. <https://doi.org/10.1109/ICCE56470.2023.10043462>

Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8.^a ed.). Pearson.

Maymounkov, P., & Mazières, D. (2002). Kademlia: A peer-to-peer information system based on the XOR metric. En P. Druschel, F. Kaashoek & A. Rowstron (Eds.), *Peer-to-Peer Systems — IPTPS 2002. Lecture Notes in Computer Science*, vol. 2429 (pp. 53–65). Springer. https://doi.org/10.1007/3-540-45748-8_5

Monteiro, J., Costa, P. A., & Leitão, J. (2022). Enriching Kademlia by partitioning. En *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)* (pp. 1167–1177). IEEE. <https://doi.org/10.1109/ICDCS54860.2022.00092>

Naik, A. R., & Keshavamurthy, B. N. (2020). Next level peer-to-peer overlay networks under high churns: A survey. *Peer-to-Peer Networking and Applications*, 13(6), 1860–1891. <https://doi.org/10.1007/s12083-020-00905-6>

Ripeanu, M., Foster, I., & Iamnitchi, A. (2002). Mapping the Gnutella network: Properties of large-scale peer-to-peer systems and implications for system design. *IEEE Internet Computing*, 6(1), 50–57. <https://doi.org/10.1109/4236.978369>

Trautwein, D., Rüti, A., Ascigil, O., Kärkäinen, L., Bajpai, V., Singla, A., & Tyson, G. (2022). Design and evaluation of IPFS: A storage layer for the decentralized web. En *Proceedings of the ACM SIGCOMM 2022 Conference* (pp. 739–752). ACM. <https://doi.org/10.1145/3544216.3544232>

10. Anexos

Anexo A — Estructura del repositorio y código fuente

El repositorio GitHub público se encuentra en [Kademlia-vs-Flooding](#) e incluye el código fuente completo, los datos de simulación en CSV y los scripts de análisis.

```
Kademlia-vs-Flooding/
├── README.md           ← instrucciones de instalación y ejecución
├── requirements.txt    ← dependencias Python
├── configs/
│   └── simulation_config.yaml ← parámetros por defecto de todas las simulaciones
├── src/
│   ├── node.py        ← clase Node con hash, igualdad y recursos
│   ├── network.py     ← topología aleatoria y distribución de recursos
│   ├── flooding.py    ← FloodingNetwork: búsqueda BFS con TTL
│   ├── kademlia.py    ← KademliaNetwork: k-buckets y lookup iterativo
│   ├── simulation.py  ← CLI argparse: --mode, --nodes, --bits, --runs
│   └── metrics.py     ← SearchMetrics, MetricsCollector, export CSV
├── data/
│   ├── flooding/      ← results_N{n}_K{k}.csv (100 repeticiones c/u)
│   ├── kademlia/      ← results_N{n}_B{b}.csv
│   └── churn/         ← flooding_N{n}_K{k}_churn{r}.csv,
│                       kademlia_N{n}_B{b}_churn{r}.csv
├── analysis/
│   ├── plot_results.py ← genera todas las figuras en informe/figures/
│   ├── statistical_analysis.py ← regresiones, IC 95%, tablas comparativas
│   └── validate_theory.ipynb ← notebook interactivo de validación
├── informe/
│   ├── TPFinal_SOR2_2026_1s_AvalosLautaro ← este documento
│   └── figures/      ← gráficos generados por plot_results.py
```

Anexo B — Instrucciones de ejecución

Instalación:

```
python -m venv venv
source venv/Scripts/activate # Windows (Git Bash)
# source venv/bin/activate   # Linux / macOS
pip install -r requirements.txt
```

Ejecución de simulaciones:

```
# Flooding -- serie completa
for N in 10 50 100 500 1000 5000 15000; do
    python -m src.simulation --mode flooding --nodes $N --neighbors 10
    --runs 100
done
```

```
# Kademia -- serie completa
for N in 10 50 100 500 1000 5000 15000; do
    python -m src.simulation --mode kademia --nodes $N --bits 16
--runs 100
done

# Churn -- flooding
for R in 0.05 0.10 0.20; do
    python -m src.simulation --mode churn --architecture flooding \
        --nodes 1000 --neighbors 10 --churn-rate $R --rounds 20
--searches-per-round 50
done

# Churn -- Kademia
for R in 0.05 0.10 0.20; do
    python -m src.simulation --mode churn --architecture kademia \
        --nodes 1000 --bits 16 --churn-rate $R --rounds 20
--searches-per-round 50
done
```

Análisis y gráficos:

```
python analysis/plot_results.py --input data/ --output informe/figures/
python analysis/statistical_analysis.py --input data/ --output data/
jupyter notebook analysis/validate_theory.ipynb
```

Tests unitarios:

```
python -m pytest tests/ -v
```

Anexo C — Análisis de sensibilidad de parámetros

Sensibilidad K en flooding

Se evaluaron $K = 5$, $K = 10$ y $K = 20$ con $N = 1.000$ y 100 repeticiones.

Tabla A1. Sensibilidad del flooding al número de vecinos K .

K	Mensajes (media)	Saltos (media)	Tasa de éxito
5	817	4,73	100%

10	911	3,34	100%
20	941	2,62	100%

K = 10 ofrece el equilibrio óptimo: K = 5 reduce mensajes pero aumenta latencia; K = 20 apenas reduce latencia a costa de un 3,3% más de mensajes y rendimientos decrecientes.

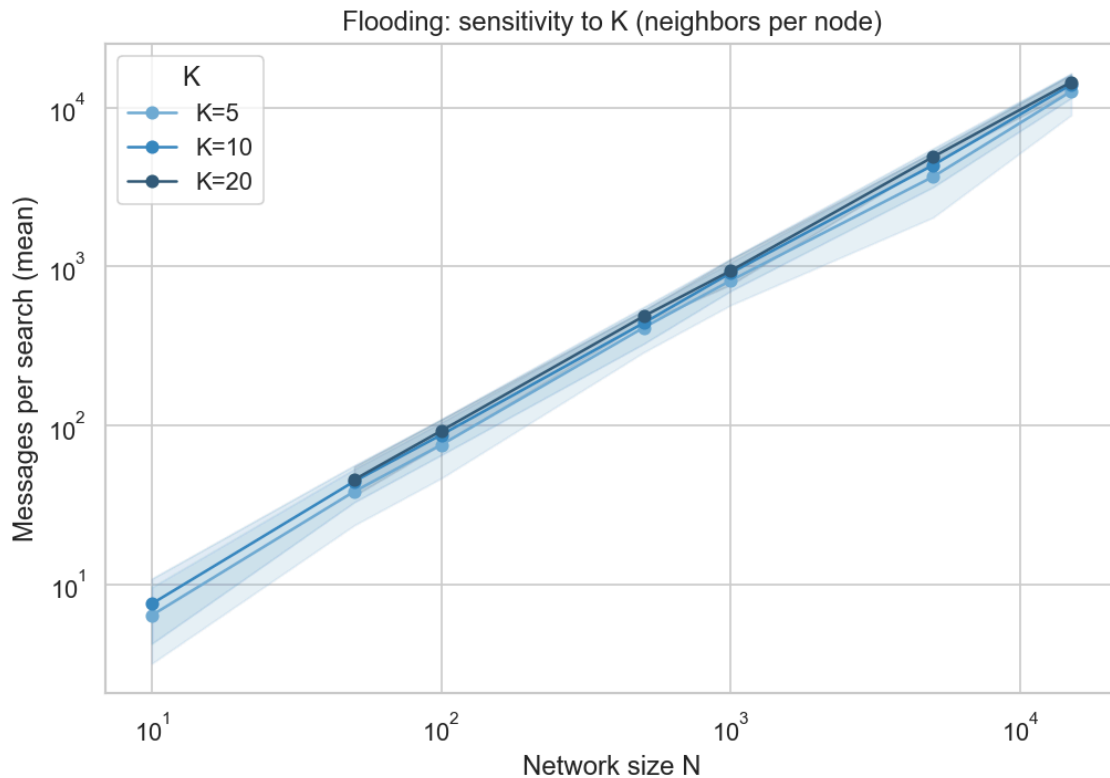


Figura A1. Sensibilidad de flooding a K. Fuente: elaboración propia.

Saturación del espacio de IDs con B = 8

Con B = 8 (espacio de 256 IDs) y N > 256 nodos, en promedio ≈ 4 nodos comparten el mismo node_id. El algoritmo de ruteo llega a un nodo con el ID correcto pero que no almacena el recurso, produciendo tasas de éxito de $\approx 10\%$ incluso sin churn. Este resultado se conserva como caso de estudio: ilustra cómo la saturación del espacio de IDs destruye el ruteo XOR antes que el churn mismo.

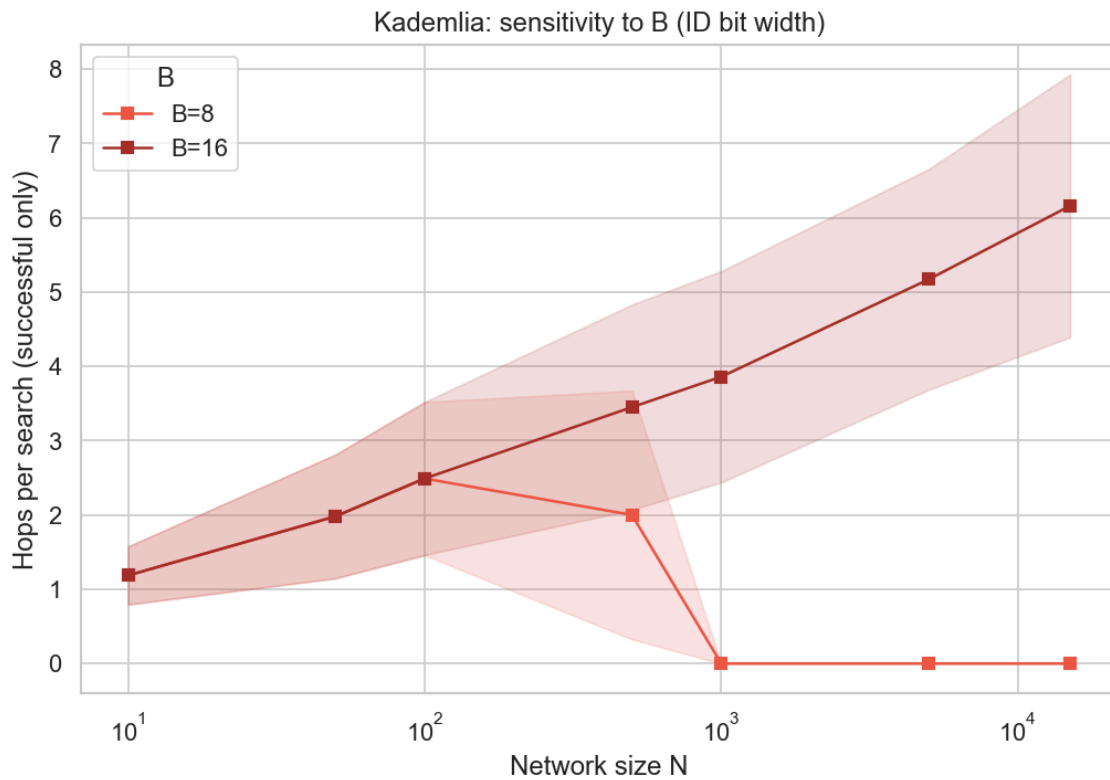


Figura A2. Sensibilidad de Kademlia a B. Fuente: elaboración propia.

Distribución de carga por nodo

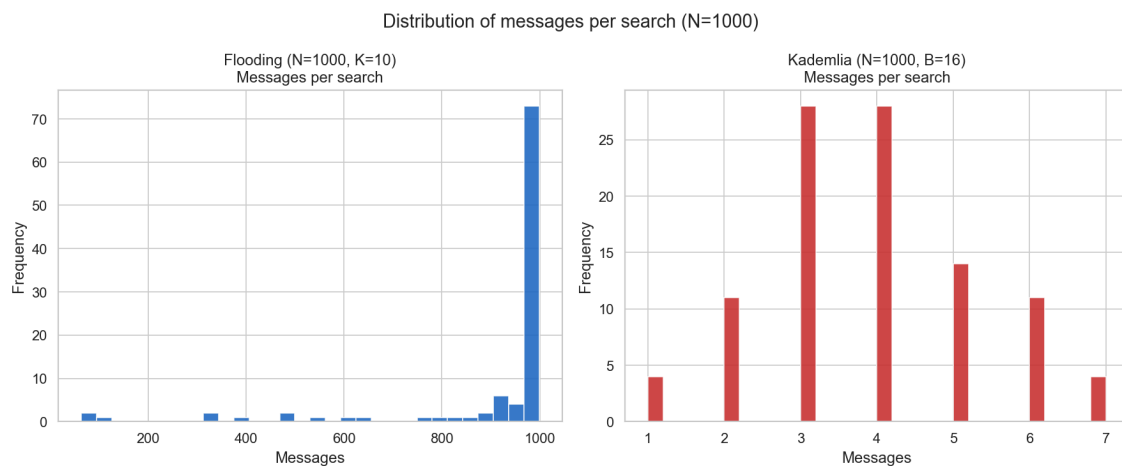


Figura A3. Distribución de mensajes por nodo en flooding ($N = 1.000$). Fuente: elaboración propia.

El histograma de la Figura A3 confirma que el flooding genera una distribución de carga altamente asimétrica: los nodos con más conexiones (hubs del grafo aleatorio) concentran una fracción desproporcionada del tráfico. Esta heterogeneidad de carga es un problema adicional del flooding a escala, más allá del overhead total de mensajes.

Anexo D — Material de presentación

Las diapositivas utilizadas para la defensa oral del trabajo se encuentran disponibles en el siguiente enlace:

 [TP_Final_SOR2_D1_Slides.pptx](#)

Las diapositivas incluyen los gráficos de resultados, la comparación flooding vs Kademia, la analogía Tcs/Tp2p y el análisis de churn.